DAY 13 – EVENING

React + TypeScript Integration • MUI Setup • CRUD UI Layout

Lecture-based study notes

Prepared from the uploaded lecture transcript and your notes-generation prompt.

Topics covered

- Creating a React + TypeScript project with Vite and understanding TS configuration
- Typing component props, state, object state and API response data
- Understanding any, JSON-to-TypeScript conversion and type naming conventions
- Choosing a UI library: MUI, Tailwind CSS, Bootstrap and related options
- Starting a CRUD UI project and installing MUI, icons and React Router
- Building an AppBar and Drawer, controlling visibility with state
- Styling with the MUI sx prop, z-index, overflow and spacing
- Rendering drawer items from an array using map()
- Separating reusable components and identifying the callback-sharing problem
- MUI theme, upcoming API integration and project-work guidance

Session transcript metadata: lecture dated Nov 2; notes named Day 13 Evening as requested.

1. Lecture roadmap

The lecture moves from making React code type-safe to preparing the visual foundation of a CRUD frontend. The first part explains how TypeScript types are assigned to React props, state and server responses. The second part introduces Material UI (MUI) and begins a reusable application shell containing a top AppBar and a side Drawer.

Phase	What you learn	Why it matters
React + TypeScript setup	Vite TypeScript variant, TSX, tsconfig files	Start with a configured TypeScript React project
Typing data	Props, state, object shapes, API response	Catch incorrect data usage while coding
UI library choice	MUI compared with utility CSS and Bootstrap	Avoid building every UI element from scratch
CRUD UI shell	AppBar, Drawer, icons, layout	Create a consistent layout for multiple screens
Reusable components	Move top bar and side navigation out of App	Keep the main component clean and reusable

Important boundary: The transcript ends while the instructor is setting up how the Drawer's open handler should be passed to the separate top-bar component. The final prop/callback solution is not demonstrated in this session, so it is marked as the next-session problem rather than presented as something completed.

2. Creating a React + TypeScript application with Vite

2.1 Why TypeScript in a React application?

The instructor explains that the earlier React code was written in JavaScript and therefore did not provide the same level of type safety. In a real application, frontend and backend code both benefit from clear data types. When using TypeScript with React, assign types to the data used by components: props, state, function arguments and function return values.

The application still behaves like a React application. The main change is the starter template and the requirement to describe the shapes of the data being used.

2.2 Create the project

```
npm create vite@latest
# or, as demonstrated in the lecture:
npx create-vite@latest

# Select:
# Project name: React TypeScript Demo
# Framework: React
# Variant: TypeScript

cd React-TypeScript-Demo
npm install
npm run dev
```

Vite asks for a framework and a variant. Choose React, then choose the TypeScript variant rather than JavaScript. Vite supplies the TypeScript configuration, so the instructor does not manually configure the compiler from scratch.

2.3 TSX instead of JSX

Extension	Used for
.jsx	JavaScript file containing JSX markup
.tsx	TypeScript file containing JSX markup
.ts	TypeScript file without JSX markup, often used for types and utility logic

A React component written with TypeScript and JSX is commonly saved with the .tsx extension. The lecture uses app.tsx and later creates a types.ts file for shared type definitions.

2.4 Understanding the TypeScript configuration files

The generated project contains tsconfig.json and, in the Vite template shown, separate configuration files such as tsconfig.app.json and tsconfig.node.json. The instructor describes tsconfig.json as the entry point that combines/references the more specific configurations.

• **tsconfig.json:** the main TypeScript project configuration entry.

• **tsconfig.app.json:** application-side TypeScript compiler settings.

• **tsconfig.node.json:** settings for Node-side configuration files and tooling.

• The configuration controls how the TypeScript compiler checks and processes the project.

The instructor's practical advice is to use the generated configuration as supplied at this stage rather than changing compiler options without a reason.

2.5 Dependency version mismatch observed in class

During npm install, the class encountered a peer-dependency/version mismatch involving the TypeScript version and the TypeScript ESLint tooling. The instructor changed the TypeScript version to 6.0.3 and retried installation; the terminal later completed the install. The transcript also mentions that the VS Code terminal appeared stuck even though installation had succeeded.

Do not treat 6.0.3 as a universal rule. This is the version workaround used in that recorded environment. In another project, check the compatibility requirements of the installed TypeScript ESLint packages and choose mutually compatible versions.

3. Typing React component props

3.1 Why do props need a type?

Props are values passed from a parent component to a child component. In the lecture's Intro component, the parent supplies the content for a heading, a description paragraph and a button. The child component should not guess the shape or data type of those values. TypeScript lets us define the expected structure.

3.2 Define an interface in a separate file

```
// types.ts
export interface IntroProps {
  headingText: string;
  description: string;
  buttonText: string;
}
```

The interface describes the contract for the component: all three properties must exist and each must be a string. The type is exported so another file can import it.

3.3 Use the interface in a component

```
// Intro.tsx
import type { IntroProps } from "./types";

export function Intro(props: IntroProps) {
  return (
    <div>
      <h1>{props.headingText}</h1>
      <p>{props.description}</p>
      <button>{props.buttonText}</button>
    </div>
  );
}
```

The component parameter is typed as IntroProps. TypeScript can now check property names and values. For example, passing a number as headingText or misspelling description will produce a type error.

3.4 Destructuring props

```
import type { IntroProps } from "./types";

export function Intro({
  headingText,
  description,
  buttonText
}: IntroProps) {
  return (
    <div>
      <h1>{headingText}</h1>
      <p>{description}</p>
      <button>{buttonText}</button>
    </div>
  );
}
```

The instructor discusses destructuring props directly in the function parameter. This changes how the values are accessed, not the type contract. The properties are read as local variables instead of using props.headingText, etc.

3.5 Where should types be stored?

The lecture demonstrates a separate types.ts file. In a larger project, the instructor recommends organizing types separately and, as the project grows, keeping type files associated with the relevant component or feature. The goal is to make the data contracts easy to find and maintain.

Naming convention mentioned in class: prefix interface names with I and type-alias names with T. This is the instructor's convention for this course; TypeScript itself does not require these prefixes.

4. Typing state with useState

4.1 Primitive state

The type argument is written in angle brackets after useState and before its initial value. It tells TypeScript what kind of value the state is allowed to hold.

```
import { useState } from "react";

const [score, setScore] = useState<number>(0);

function updateScore() {
  setScore(3);       // valid
  // setScore("3");  // TypeScript error
}
```

Because the state is declared as number, setScore accepts numeric values. A string cannot be assigned to it. This is the type-safety benefit demonstrated by the instructor.

4.2 Common primitive state types

```
const [count, setCount] = useState<number>(0);
const [userName, setUserName] = useState<string>("");
const [isOpen, setIsOpen] = useState<boolean>(false);
```

These examples illustrate the same generic syntax. The type should match the kind of data the state is intended to hold.

4.3 Object state

A form state can contain multiple key-value pairs. The lecture's example includes name, phone number, email and password. Since this is a custom object shape, define an interface or a type alias first.

```
// types.ts
export interface IFormDataState {
  name: string;
  phoneNumber: string;
  email: string;
  password: string;
}

// In a React component
const [formData, setFormData] =
  useState<IFormDataState>({
    name: "",
    phoneNumber: "",
    email: "",
    password: ""
  });
```

The generic type IFormDataState constrains the state object. The object must follow the declared structure. The lecture's key idea is that a state can hold a primitive value or a custom object, and both can be typed.

Beginner check: The type argument does not create the object. It describes the expected shape. The initial value still supplies the actual starting object.

5. Typing functions and API response data

5.1 Function arguments and return types

The instructor states that functions also need type information: their arguments should have types and their return values should be described where applicable. The transcript focuses more on the idea than on a complete separate function-typing example.

```
function add(a: number, b: number): number {
  return a + b;
}

const showMessage = (message: string): void => {
  console.log(message);
};
```

In the examples above, a and b are numbers, add returns a number, message is a string, and void indicates that showMessage does not return a useful value. These examples illustrate the function-typing principle stated in the lecture.

5.2 Fetching data: response versus JSON data

```
async function fetchUserData() {
  try {
    const response = await fetch("API_URL");
    const userData = await response.json();

    console.log(userData);
  } catch (error) {
    console.log(error);
  }
}
```

The lecture walks through this sequence:

• fetch() makes the request and returns a Promise.

• await waits for the fetch Promise to resolve to a Response object.

• response.json() reads/parses the response body and itself returns a Promise.

• await waits for that Promise and assigns the parsed result to userData.

• The catch block handles an error thrown during the operation.

The Response type is already known to TypeScript because fetch has a predefined type. However, the JSON method cannot know whether the server will return users, products, employees or another structure.

5.3 What does any mean?

The instructor explains that JSON data is treated as any in the demonstrated typing context because the built-in JSON parsing method cannot know the application-specific structure of the server's response. any allows values of different types to be assigned without type checking. This means type safety and useful property suggestions are lost.

Type	Meaning	Effect
any	Disables type checking for that value	Flexible, but unsafe; property suggestions are not reliable
A defined interface/type	Describes the expected data structure	TypeScript can check property access and provide suggestions

Instructor's warning: Avoid any wherever possible. Use it only when there is a genuine reason, because it weakens the type safety that TypeScript is intended to provide.

5.4 Describe the API response shape

The lecture's sample user response is an array of objects. Each user object has properties such as id and name, and nested objects such as address and company. The frontend should create types that represent the response shape.

```
interface IAddress {
  street: string;
  city: string;
  zipcode: string;
}

interface ICompany {
  name: string;
}

interface IUser {
  id: number;
  name: string;
  username: string;
  email: string;
  address: IAddress;
  company: ICompany;
}
```

This is a simplified teaching shape based on the fields discussed in the transcript. A real interface must match the actual API response, including every field the application intends to use.

5.5 Annotate the parsed JSON

```
async function fetchUserData() {
  try {
    const response = await fetch("API_URL");
    const userData = await response.json() as IUser[];

    userData.map((user) => {
      console.log(user.address.city);
      console.log(user.company.name);
    });
  } catch (error) {
    console.log(error);
  }
}
```

The as IUser[] expression tells TypeScript to treat the parsed value as an array of IUser objects. It allows the editor to suggest properties such as address, company and their nested members.

Accuracy note: A type assertion does not validate the server's JSON at runtime. It tells the TypeScript compiler what shape the developer expects. The server response must actually match that shape.

5.6 JSON to TypeScript conversion tool

The instructor demonstrates searching for "JSON to TS" and using transform.tools. The workflow is:

• Copy a representative JSON response from the backend/API.

• Paste it into the JSON-to-TypeScript converter.

• Review the generated interfaces/types, including nested objects.

• Copy the generated definitions into the project's type file.

• Rename generated types according to the project's naming convention, such as IUser and ICompany.

• Use the resulting type for the parsed API data.

The converter saves time when a response contains many nested fields. Still, inspect the generated output: optional fields, null values, arrays and inconsistent sample data may need careful handling.

6. Choosing a UI approach for React

6.1 Plain CSS

The instructor notes that developers can style the whole interface with CSS, but building and maintaining every element manually takes time. The session therefore considers existing styling systems and component libraries.

6.2 Tailwind CSS

Tailwind is described as a utility-first CSS framework/library. It supplies utility classes for common styling tasks such as spacing, alignment, colors, borders and other visual adjustments. It does not primarily provide a complete set of ready-made application components in the way a component library does; developers compose the interface themselves.

6.3 Bootstrap

Bootstrap is described as providing predefined UI elements and responsive layout support. Examples mentioned include forms, tables, navigation bars, side navigation and popup dialogs.

6.4 MUI (Material UI)

The instructor chooses MUI for the React project because it provides ready-made React components and a broad set of customizations. Examples discussed include buttons, checkboxes, rating controls, dropdowns, sliders, switches, input fields, avatars, icons, alerts/dialogs, progress indicators, skeleton loaders, AppBar and Drawer.

Option	Main idea in this lecture	Typical use
Plain CSS	Write and maintain styles yourself	Full manual styling control
Tailwind CSS	Utility classes; compose the design	Spacing, colors, borders and layout utilities
Bootstrap	Predefined UI patterns and responsive helpers	Forms, tables, navbars, dialogs
MUI	Ready-made React UI components	React application interface and reusable controls
DaisyUI	Mentioned as a component layer built on Tailwind CSS	Pre-styled components using Tailwind
Ant Design	Mentioned as another UI library option	Alternative component library
Angular Material / PrimeNG	Angular-oriented component options	Angular applications, not the React MUI components used here

The instructor emphasizes that MUI components are React components. Angular has its own component ecosystem, including Angular Material and PrimeNG. The lecture's selection is MUI because of its component range and community.

6.5 Documentation-first workflow

The instructor says not to memorize every component's code. Instead, open the official documentation, find the required component, inspect its example and properties, and adapt the code to the application. The documentation can also provide a sandbox where the output can be previewed and customized.

This is an important practical habit: learn what a component does and what its props control, rather than trying to memorize every library API.

7. Starting the CRUD UI project

7.1 Project purpose

The class previously created a CRUD API for product data. The current session begins the matching frontend project, named CRUD UI. The frontend will later be expanded through multiple sessions and connected to the backend.

```
npm create vite@latest
# Project name: CRUD UI
# Framework: React
# Variant: TypeScript

cd CRUD-UI
npm install
```

7.2 Dependencies discussed

The transcript installs the MUI dependencies, the MUI Material Icons package and React Router DOM. The standard package names for the libraries discussed are shown below as a study reference.

```
npm install @mui/material @emotion/react @emotion/styled
npm install @mui/icons-material
npm install react-router-dom

npm run dev
```

MUI uses Emotion as its styling engine in the standard setup shown by its installation instructions. The icon package is separate: without it, the project cannot import the Material icons used in the lecture.

The instructor again encounters a TypeScript/ESLint peer-dependency mismatch and applies the same version adjustment used earlier in class. As noted above, use compatible versions for the current project rather than blindly copying a version number.

7.3 Initial application files

The instructor clears the default starter content from app.tsx and removes the default index.css import while preparing the new layout. The intention is to start with a clean application shell. The transcript shows the app component as the first application component rendered.

React Router DOM is installed in preparation for routing. The instructor wraps the application layout with BrowserRouter as the project begins to deal with routes. Detailed route definitions are not taught in this portion.

8. Designing the shared application layout

8.1 Plan the layout before selecting components

The instructor first sketches the desired screen, then chooses MUI components. The layout is intended to remain consistent across screens such as Dashboard, About and Contact. The page-specific content may change, but the application shell should stay the same.

• A top AppBar / navbar.

• A menu icon button on the left.

• A brand/application title beside the menu icon.

• A Logout button on the right.

• A side Drawer that opens when the menu icon is clicked.

• The Drawer contains navigation items such as Dashboard, Create Product and Products List.

The key design principle is that the top bar and side navigation belong to the common application layout, not to one individual page.

8.2 MUI components used in the AppBar

Component	Role in the layout
Box	Outer wrapper/container
AppBar	Top application bar surface
Toolbar	Arranges the controls inside the AppBar
IconButton	Makes the menu icon clickable
MenuIcon	The three-line menu symbol
Typography	Displays the application title as styled text
Button	Displays the Logout action

8.3 AppBar structure – illustrative code

```
import {
  AppBar, Box, Toolbar, IconButton,
  Typography, Button
} from "@mui/material";
import MenuIcon from "@mui/icons-material/Menu";

function ApplicationTopBar() {
  return (
    <Box>
      <AppBar position="fixed">
        <Toolbar>
          <IconButton color="inherit" edge="start">
            <MenuIcon />
          </IconButton>

          <Typography variant="h6" component="div">
            CRUD UI
          </Typography>

          <Button color="inherit">
            Logout
          </Button>
        </Toolbar>
      </AppBar>
    </Box>
  );
}
```

This code illustrates the component hierarchy taught in class. In the actual project, the menu button later needs an onClick function so it can open the Drawer. The Logout button is part of the visual shell at this stage; logout behaviour is not implemented in the transcript.

8.4 MUI components versus HTML tags

The instructor points out that the capitalized Button imported from MUI is not the same as the lowercase HTML button element. MUI's Button is a React component with MUI styling and props. Similarly, Typography is used for text presentation. The instructor recommends using MUI components consistently when building a screen with MUI, while using native HTML elements only when there is a reason.

8.5 The sx prop

For styling MUI components, the lecture uses the sx prop rather than the HTML style attribute. The sx prop accepts style values in an object and supports MUI's styling system.

```
<IconButton sx={{ mr: 2 }}>
  <MenuIcon />
</IconButton>

<Typography sx={{ mt: 1, mb: 2 }}>
  CRUD UI
</Typography>
```

The instructor explains mr as margin-right and mentions related spacing shorthands such as mt and mb. The exact numeric spacing interpretation follows MUI's theme spacing system; it should not automatically be read as a percentage.

8.6 Run and verify

```
npm run dev
```

The Vite development server in the lecture runs on port 5173. The browser shows the AppBar with the menu icon, CRUD UI title and Logout button.

9. Drawer: visibility controlled by state

9.1 What is a Drawer?

A Drawer is a side panel used for navigation. MUI supplies a Drawer component. The lecture's intended interaction is that the Drawer is initially hidden and becomes visible when the user clicks the menu icon.

9.2 The open prop

The Drawer's open prop controls its visibility: true means open; false means closed. This is a natural use case for React state because the value changes in response to user interaction.

```
import { useState } from "react";
import { Drawer, IconButton } from "@mui/material";

function App() {
  const [openDrawer, setOpenDrawer] = useState(false);

  return (
    <>
      <IconButton onClick={() => setOpenDrawer(true)}>
        <MenuIcon />
      </IconButton>

      <Drawer
        open={openDrawer}
        onClose={() => setOpenDrawer(false)}
      >
        {/* Drawer content */}
      </Drawer>
    </>
  );
}
```

Execution flow:

• The component first renders with openDrawer = false.

• The Drawer receives open={false}, so it is hidden.

• The user clicks the menu IconButton.

• The click handler calls setOpenDrawer(true).

• React updates state and re-renders the component.

• The Drawer now receives open={true} and becomes visible.

• When the Drawer is dismissed, onClose sets the state back to false.

The transcript explicitly teaches that a state update causes a re-render and that the new state value controls the Drawer's visibility.

9.3 BrowserRouter

The instructor wraps the application shell with BrowserRouter as routing will be used throughout the application. The transcript does not yet show route definitions or page navigation. BrowserRouter is therefore a setup step in this lecture, not a complete routing lesson.

```
import { BrowserRouter } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      {/* Shared application layout and routed content */}
    </BrowserRouter>
  );
}
```

10. Drawer content, icons and styling

10.1 Drawer list structure

The instructor uses MUI's Drawer together with List, ListItemButton, ListItemIcon and ListItemText. Each menu row is a clickable list item. Its icon is rendered inside ListItemIcon and its label inside ListItemText.

MUI component	Purpose
Drawer	Side panel container
List	Groups the navigation entries
ListItemButton	Makes an entry behave like a clickable row
ListItemIcon	Holds the icon for the entry
ListItemText	Displays the entry label

10.2 Icons

The class uses the @mui/icons-material package and searches the MUI icon catalogue to find suitable icons. The examples include Dashboard, Add and List Alt icons for the three menu entries. The instructor notes that icon selection is done through the documentation/icon list rather than memorizing icon names.

10.3 z-index: control which element appears above another

The Drawer initially appears above the AppBar. The instructor wants the AppBar to remain above the Drawer, so the lecture introduces the CSS z-index property. z-index controls stacking order for elements that participate in the relevant stacking context.

Rather than hardcoding an arbitrary value, the instructor demonstrates reading the Drawer z-index from the MUI theme and adding one to it. This makes the AppBar's z-index one level higher than the Drawer's theme value.

```
// Illustrative MUI sx usage on AppBar
<AppBar
  sx={(theme) => ({
    zIndex: theme.zIndex.drawer + 1
  })}
>
  {/* toolbar */}
</AppBar>
```

The transcript records a debugging step: the first attempt placed sx on the wrong wrapper and did not produce the desired stacking. Moving the sx styling to the AppBar itself and reloading resolved the layering issue.

10.4 Drawer content hidden behind the AppBar

After placing the AppBar above the Drawer, the top of the Drawer content is hidden behind the AppBar. The instructor notices that the Home/navigation area is no longer visible and adjusts the Drawer content's top spacing.

```
<Box sx={{ pt: 7, overflow: "auto" }}>
  {/* Drawer list */}
</Box>
```

The lecture experiments with top padding (first 30px and then 60px) to move the content below the fixed AppBar. It also introduces overflow: auto so that if the menu grows beyond the available browser height, the content can scroll.

Remember: z-index controls stacking; padding creates space inside an element; overflow controls what happens when content exceeds its available area. They solve different problems.

11. Making Drawer items data-driven with map()

11.1 The problem with repeated markup

At first, each navigation entry is written separately in JSX. This repeats almost identical component structure: a list-item button, an icon and a text label. The only things that change are the icon and label. In a real application there may be 10–20 or more menu items, so repeating the same markup becomes difficult to maintain.

11.2 Store the menu data in an array

The instructor replaces repeated markup with an array of objects. Each object stores the text and icon for one menu item. The menu can then be extended by adding another object to the array.

```
const drawerMenu = [
  {
    text: "Dashboard",
    icon: <DashboardIcon />
  },
  {
    text: "Create Product",
    icon: <AddIcon />
  },
  {
    text: "Products List",
    icon: <ListAltIcon />
  }
];
```

11.3 Render the array using map()

```
<List>
  {drawerMenu.map((menuItem) => (
    <ListItemButton key={menuItem.text}>
      <ListItemIcon>
        {menuItem.icon}
      </ListItemIcon>

      <ListItemText primary={menuItem.text} />
    </ListItemButton>
  ))}
</List>
```

map() visits each array element and returns a new array of JSX elements. For every menuItem, the callback returns one ListItemButton. The icon comes from menuItem.icon and the label comes from menuItem.text.

• The array stores the data.

• map() loops over the array and calls the callback for each element.

• The callback returns the JSX structure for one menu row.

• Adding a new object to drawerMenu adds another rendered row without duplicating the JSX structure.

The key prop is included in the illustrative code to identify each rendered row. The transcript's main teaching point is the data-driven rendering pattern: one reusable structure, many data entries.

12. Component separation and the unfinished callback-sharing problem

12.1 Why split the App component?

The instructor points out that writing all AppBar and Drawer code inside App makes the main component lengthy and difficult to manage. These two pieces do not represent individual screens; they are reusable application-shell components.

The proposed folder structure is:

```
src/
```

```
components/
  ApplicationTopBar.tsx
  SideNav.tsx
App.tsx
main.tsx
types.ts
```

Move the AppBar code into ApplicationTopBar.tsx and the Drawer code into SideNav.tsx. Import the required MUI components in the files where they are used.

12.2 Which state belongs where?

The instructor starts moving the Drawer state and its related functions into SideNav. The menu data also belongs with the side-navigation implementation because it describes the Drawer's content.

However, a design issue appears: the function that opens the Drawer is needed by the menu icon in ApplicationTopBar, while the close handler is needed by the Drawer itself. The instructor asks how the open handler can be passed into ApplicationTopBar. The lecture stops before showing the solution.

12.3 Understand the requirement

Item	Needs to use it
handleOpenDrawer	ApplicationTopBar – because the menu icon is there
handleCloseDrawer	SideNav / Drawer – because the Drawer closes from there
openDrawer state	Controls whether the Drawer is open or closed
drawerMenu array	Supplies the navigation entries to SideNav

The key concept to investigate next is passing a function from one component to another through props. In a common React design, the component that owns the shared state passes the relevant handler down as a prop to the component that needs to call it. The transcript does not complete this wiring, so treat it as a preparation task for the next lecture.

```
// Illustrative pattern only: parent owns state and passes a callback
function App() {
  const [open, setOpen] = useState(false);

  return (
    <>
      <ApplicationTopBar onMenuClick={() => setOpen(true)} />
      <SideNav
        open={open}
        onClose={() => setOpen(false)}
      />
    </>
  );
}
```

This parent-owned-state example is supplementary clarification of the problem raised at the end of the lecture; it is not presented as the instructor's completed implementation.

13. MUI themes and dark/light mode

A student asks whether dark mode and light mode are handled automatically. The instructor explains that MUI has built-in theme support. The developer defines the theme, including the dark or light palette, and the UI can respond to the selected theme. The instructor suggests searching for "MUI theme" to study the concept.

A theme is a shared configuration that can control visual decisions across components. The transcript mentions the theme concept and the z-index values available through theme, but it does not implement a full dark/light toggle in this session.

14. What comes next: connecting frontend and backend

The instructor says upcoming sessions will focus on connecting the frontend built in React to the backend and calling APIs from the frontend application. The current lecture lays the groundwork: a typed React application, API response interfaces, a CRUD UI shell and navigation.

The project discussion at the end also advises students to prepare generalized backend functionality for login and signup, database work and then adapt it to the assigned project topic. Students should understand the complete functionality across frontend, backend and database rather than limiting themselves to one technology or only building static pages.

• Understand the project's required functionality before dividing work.

• Get exposure to frontend, backend and database integration.

• Prepare common login/signup functionality where appropriate.

• Be ready to explain what you implemented during project evaluation.

• Do not limit project work to static pages such as only Home or Contact.

15. Quick revision tables

15.1 TypeScript + React

Concept	Remember
TSX	TypeScript file that contains JSX
tsconfig.json	Controls TypeScript compiler/project configuration
Props	Data passed into a component
Interface	Describes the expected object/property structure
useState(0)	Creates numeric state
useState(...)	Creates state constrained to a custom object shape
any	Allows arbitrary values and weakens type safety
as IUser[]	Type assertion for the expected API response shape
map()	Transforms each array entry into a returned value, here JSX

15.2 MUI + layout

Concept	Remember
AppBar	Top application bar
Toolbar	Arranges controls inside the AppBar
IconButton	Clickable icon control
Typography	MUI text component
Drawer open prop	true shows; false hides
sx	MUI styling prop
z-index	Controls stacking order
overflow: auto	Allows scrolling when content overflows
ListItemIcon / ListItemText	Icon and label in a navigation row
BrowserRouter	Router wrapper prepared for client-side routing
Theme	Shared design configuration, including z-index and palette

16. Common mistakes and troubleshooting

• **Using .jsx for a TypeScript component:** use .tsx when the file contains TypeScript and JSX.

• **Leaving props untyped:** define an interface/type and use it in the component parameter.

• **Assigning the wrong value to state:** match setState values to the declared state type.

• **Using any for every API response:** define the response shape so property access can be checked.

• **Assuming JSON parsing knows the response shape:** the parser cannot infer whether the payload is a user, product or another entity.

• **Confusing MUI Button with HTML button:** the capitalized imported component is MUI; lowercase button is the native HTML element.

• **Expecting the Drawer to open by itself:** control its open prop with state and update that state from an event handler.

• **Hardcoding z-index without checking the theme:** the lecture derives AppBar z-index from theme.zIndex.drawer + 1.

• **Using margin to solve stacking:** margin/padding and z-index solve different layout problems.

• **Repeating every navigation row manually:** store entries in an array and render them with map().

• **Keeping every component inside App:** move reusable AppBar and Drawer code into separate component files.

• **Assuming the final callback wiring was completed:** the transcript ends with this as an open question for the next session.

17. Interview and MCQ preparation

Q. Why use TypeScript with React?
To describe data shapes and catch invalid prop, state and function usage during development.

Q. What is the difference between .ts and .tsx?
Use .tsx when the TypeScript file contains JSX markup; .ts is for TypeScript without JSX.

Q. Why define an interface for props?
It specifies which properties a component expects and their data types.

Q. How do you type a numeric state?
useState<number>(initialNumber).

Q. What is the problem with any?
It allows arbitrary values and removes much of TypeScript's checking and editor assistance.

Q. Why does response.json() not know the application data shape?
The JSON parser is generic; it cannot know the backend's domain model from the method call alone.

Q. What does as IUser[] do?
It asserts to TypeScript that the value is expected to be an array of IUser objects; it does not validate runtime data.

Q. What is Tailwind's approach in this lecture?
Utility-first classes used to compose styling.

Q. What is MUI?
A React component library based on Material UI design principles.

Q. What does the Drawer open prop control?
Its visibility; true opens it and false closes it.

Q. Why use useState for the Drawer?
The visibility changes in response to user interaction and should trigger a re-render.

Q. What is the purpose of sx?
To apply styling to MUI components using the MUI styling system.

Q. What does z-index do?
Controls the stacking order of overlapping elements.

Q. Why use map() for navigation items?
It renders a repeated JSX structure from an array of data.

Q. Why separate ApplicationTopBar and SideNav?
They are reusable shell components and separating them keeps App cleaner.

Q. What issue remained unresolved at the end?
How to pass the Drawer open handler to the separate ApplicationTopBar component.

18. Practice exercises

Easy

• Create a Vite React project using the TypeScript variant. Identify the .tsx and tsconfig files.

• Create an IntroProps interface with title, description and buttonLabel properties.

• Create a numeric counter state and try assigning a string to it. Observe the TypeScript error.

• Create a boolean state called isDrawerOpen with an initial value of false.

Moderate

• Create an IProduct interface with id, name and price. Type a state variable that stores an array of products.

• Create interfaces for a user object containing a nested address object.

• Take a sample API JSON response and convert it into TypeScript interfaces using a JSON-to-TypeScript tool. Review the generated types.

• Create a Drawer that opens from an IconButton and closes through onClose.

• Create three menu entries in an array and render them with map().

Challenging

• Move the AppBar and Drawer into separate component files while keeping their interaction working.

• Pass an open-handler callback to ApplicationTopBar through props.

• Use the MUI theme to keep the AppBar above the Drawer without hardcoding an unrelated z-index.

• Make a long Drawer menu scroll using overflow: auto while keeping the AppBar visible.

• Prepare a typed frontend API call for the product backend and describe the response shape.

19. Final revision checklist

■ I can create a React + TypeScript project with Vite.

■ I understand the purpose of tsconfig.json and the split app/node configurations.

■ I know when to use .ts and .tsx.

■ I can define an interface and apply it to component props.

■ I can type primitive and object state with useState().

■ I understand why any weakens type safety.

■ I can describe an API response with interfaces, including nested objects and arrays.

■ I understand that a type assertion does not perform runtime validation.

■ I can explain the difference between Tailwind, Bootstrap and MUI as presented in class.

■ I can identify the roles of AppBar, Toolbar, IconButton, Typography and Button.

■ I can control Drawer visibility using state and its open prop.

■ I understand sx, z-index, padding and overflow: auto.

■ I can render a list of menu items from an array using map().

■ I understand why reusable components should be separated from App.

■ I can explain the callback-sharing problem left for the next session.

■ I am ready to connect the React frontend to backend APIs in the upcoming sessions.

20. Scope and source-faithfulness notes

These notes follow the uploaded lecture transcript's sequence and preserve the instructor's central explanations. Irrelevant side conversations and repeated classroom filler have been removed. Where the transcript describes an implementation without providing a complete code listing, the notes supply a readable teaching example and label supplementary clarification where needed.

Topics not completed in the transcript are not presented as completed lecture work: full route configuration, logout behaviour, full dark/light theme switching, backend API integration and the final callback wiring between ApplicationTopBar and SideNav remain future work.

Requested filename: **Day 13 Evening – React TypeScript Integration, MUI Setup and CRUD UI**.